

DD: ICN-01 — Internal Communications Service

Developer implementation guide: DD_ICN-01-Internal_Communications-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
DD_ICN-01-Internal_Communications-v1.0.md.

Verdict: ■ New | **Wave:** 2 | **Group I:** Foundations | **Version:** v1.0 | **Module:** Internal Communications The system of record for **staff-facing** notification dispatch: emails, Slack messages, Microsoft Teams cards, and in-app push notifications sent to BO users (dispatchers, CSRs, technicians, NOC, Finance, KYC reviewers). Called by Camunda BPMN HTTP connectors via per-module NotificationHandlerResource wrappers when a workflow parks at a user task and needs human action. Retires staff_notification_stub in FUL-02 FRAMEWORK §6.3 and WO module FRAMEWORK §9.1. ICN-01 is **strictly staff-scoped**. Customer-facing notifications are owned by DD_NOT-01 and travel a separate path. This separation is enforced by API surface and by event-topic naming — the two modules never overlap.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/10_other/DD_ICN-01-Internal_Communications-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/staff-notification-user-channel-identity/{user_id}/{channel}
- GET /api/groups/kenya-l1-kyc/members?operatorCode=WIK
- GET /api/groups/{group}/members
- GET /api/groups/{group}/members?operatorCode=<>
- GET /api/icn/adapters-registry
- GET /api/staff-notification-adapters-bindings/{operator}/{channel}

- GET /api/staff-notification-adapter-bindings?operatorCode=WIK
- GET /api/staff-notification-channel-config?operatorCode=WIK
- GET /api/staff-notification-templates?operatorCode=WIK
- GET /api/staff-notification-user-channel-identity/{user_id}
- GET /api/staff-notification-user-channel-identity/{user_id}/{channel}
- GET /api/staff-notification-user-pref/{user_id}
- GET /api/staff-notifications/inbox?status=DISPATCHED&limit=20
- GET /api/staff-notifications/stats?operatorCode=WIK&since=2026-05-01
- GET /api/staff-notifications/{id}
- GET /api/staff-notifications/{id}/ack?recipient={userId}
- GET /api/staff-notifications?source_module=BIL-04&since=2026-05-01
- GET /api/users/{userId}
- GET /api/users/{user_id}
- POST /api/orders/{orderId}/user-tasks/{taskId}/complete
- POST /api/staff-notification-adapter-bindings/{operator}/{channel}/test
- POST /api/staff-notification-templates
- POST /api/staff-notifications
- POST /api/staff-notifications/notif_01HZ_KYC_L1_JANE/ack
- POST /api/staff-notifications/{notification_id}/ack
- POST /engine-rest/task/{id}/complete
- PUT /api/staff-notification-adapter-bindings/{operator}/{channel}
- PUT /api/staff-notification-channel-config/{operatorCode}
- PUT /api/staff-notification-templates/{operator}/{code}/{channel}/disable
- PUT /api/staff-notification-user-channel-identity/{user_id}/{channel}
- PUT /api/staff-notification-user-pref/{user_id}

4.2 Tables

- staff_notification
- staff_notification_adapter_binding
- staff_notification_channel_config
- staff_notification_delivery
- staff_notification_template
- staff_notification_user_channel_identity
- staff_notification_user_pref

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- adjustment-approval-needed

- auto-resolved-from-auth
- await-l1-kyc-approval
- aws-ses
- bounce-poller
- customer-complaint-routed-for-attention
- directory-sync
- dunning-l2-escalation-internal
- dunning-l3-escalation-internal
- email-adapter
- finance-refunds-approvers
- inapp-adapter
- inapp-firebase-cloud-messaging
- inapp-server-sent-events
- inapp-websocket-fanout
- install-wo-awaiting-dispatcher
- install-wo-awaiting-tech-claim
- install-wo-finalize-confirm-needed
- kenya-dunning-l3-reviewers
- kenya-kyc-leaders
- kenya-l1-kyc
- kyc-final-approval-needed
- kyc-l1-approval-needed
- kyc-rejected-resubmit
- mailgun-api
- microsoft-graph-mail
- msteams-adapter
- msteams-graph-channel
- msteams-incoming-webhook
- network-incident-confirmation-needed
- noc-team
- outage-credit-batch-ready
- pagerduty-events-v2
- postmark-api
- provisioning-failure-noc-investigate
- refund-approval-needed
- sendgrid-api
- shifting-disconnect-confirm
- shifting-reconnect-tech-needed
- slack-adapter
- slack-bot-api
- slack-connect-api

- smtp-classic
- sophix-superadmin
- support-wo-tech-needed-contractor
- support-wo-tech-needed-internal
- twilio-programmable-chat
- vip-customer-escalation
- whatsapp-business-api

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.

Concern	Recommendation
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group I:** Foundations | **Version:** v1.0 | **Module:** Internal Communications

The system of record for **staff-facing** notification dispatch: emails, Slack messages, Microsoft Teams cards, and in-app push notifications sent to BO users (dispatchers, CSRs, technicians, NOC, Finance, KYC reviewers). Called by Camunda BPMN HTTP connectors via per-module `NotificationHandlerResource` wrappers when a workflow parks at a user task and needs human action. Retires `staff_notification_stub` in FUL-02 FRAMEWORK §6.3 and WO module FRAMEWORK §9.1.

ICN-01 is **strictly staff-scoped**. Customer-facing notifications are owned by DD_NOT-01 and travel a separate path. This separation is enforced by API surface and by event-topic naming — the two modules never overlap.

1. What this module is

When a Camunda BPMN process parks at a user task — "L1 supervisor must approve KYC", "contractor dispatcher must assign a tech", "NOC must verify provisioning succeeded", "Finance must approve a refund" — the relevant human(s) need to be told the task exists, ideally on the channel they actually check (email today, Slack/Teams in many teams, in-app push when they're in the BO UI).

Today (v1.0 with `staff_notification_stub`) every such alert goes via internal SMTP only. That misses people who don't watch email closely and delays response. ICN-01 fans the alert out across the channels each recipient prefers, falling back across them in priority order until one acknowledges.

What this module does

1. **Receives dispatch requests** from per-module `NotificationHandlerResource` (FUL-02, WO, future SUB-WF) over HTTP.
2. **Resolves candidate groups** to individual staff users via `FOUNDATION_AUTH`'s group-membership API.
3. **Looks up each recipient's channel preferences** (which channels are enabled, priority order, working-hours, suppression).
4. **Dispatches the message** on each enabled channel (Email via internal SMTP, Slack via Slack Bot API, MS Teams via Incoming Webhook, in-app push via WebSocket fanout to BO UI sessions).
5. **Tracks delivery state** per (notification, recipient, channel): DISPATCHED / FAILED / ACKNOWLEDGED.
6. **Emits Kafka events** for delivery state changes — observability + analytics consumers.
7. **Provides template rendering** — operator-localised text + variable substitution. v1.0 ships ~30 templates (KE seed) for KYC, install dispatch, support ticket, dunning escalation, finance approvals.
8. **Handles bouncing** — failed-email bounce-back via SMTP, Slack/Teams 4xx responses, push-WebSocket disconnects. Retries with backoff up to 3 times per channel; falls through to next channel on exhaustion.
9. **Surfaces an in-app inbox** — every BO user sees their pending user-task notifications in a tray in the BO UI, deep-linked to the Camunda task in their queue.

What this module does NOT do

- **Doesn't notify customers.** Strict scope: every endpoint is staff-only. Customer notifications (SMS to subscriber, email to billing-email) live in DD_NOT-01 and are entirely separate. There is no shared code path. Architectural rule: ICN-01 never writes to a `customer_*` table, never reads `customer_contact_method`, never knows a customer's MSISDN.
- **Doesn't define candidate groups.** Group codes (kenya-l1-kyc, kenya-kyc-leaders, tech-region-NRB-KAREN, noc-team, finance-refunds-approvers, etc.) are owned by `FOUNDATION_AUTH`. ICN-01 looks up members; it does not create groups.

- **Doesn't define templates' business semantics.** Templates ship with field placeholders. The calling module decides what to put in them via the `templateVariables` payload. ICN-01 doesn't validate that the template's variables match the calling workflow.
- **Doesn't drive workflow state.** When a recipient ACKs a notification, that's an observability signal only. The actual task-completion path is the BO UI calling the owning module's `/user-tasks/{taskId}/complete` endpoint (per FUL-02 FRAMEWORK §3, WO module FRAMEWORK §6). ICN-01 is a transport layer.
- **Doesn't decide who acts.** Workflow modules choose the candidate group; FOUNDATION_AUTH defines who's in it; ICN-01 just notifies all members of that group. Priority / round-robin / load-balancing across staff members is the workflow's concern, not ICN-01's.
- **Doesn't store the canonical user task.** Camunda owns that. ICN-01 stores a reference (`camunda_task_id`) and the rendered notification — never duplicates task data.
- **Doesn't implement audit-grade non-repudiation.** Delivery confirmations are best-effort and recorded in `staff_notification_delivery`; that table is for operational observability, not legal-grade proof of receipt. Audit-grade trails go in FOUNDATION_AUDIT (separate module).
- **Doesn't expose customer-PII-bearing message bodies.** Rendered messages reference customer/account by identifier and minimal context (name on `customer.name`, account number) — not by full identification numbers, full address, or financial detail. PII minimization is enforced at template definition time.

Regional flexibility

Different operators (WIK, WUG, WTZ, YASSN) may use different mixes of channels:

- **WIK (Kenya)** — Email + Slack heavy; in-app push for dispatchers. Teams not used.
- **WUG (Uganda)** — Email + WhatsApp Business (extension future-roadmap; v1.0 ships Email + Slack + in-app push, WhatsApp deferred).
- **WTZ (Tanzania)** — Email primary; some teams use Slack.
- **YASSN (Senegal)** — Email + Microsoft Teams; Slack not in standard usage.

The schema and APIs support all four channel kinds (EMAIL, SLACK, MSTEAMS, IN_APP_PUSH) regardless of operator. Per-operator config decides which channels are enabled, the priority order, the fallback rules, and the integration endpoint configuration (Slack workspace, Teams webhook URL, SMTP relay, push WebSocket URL). KE is the v1.0 seed; UG/TZ/SN seeds are added as catalog rows + per-operator config without schema change.

Per memory rule #18 (regional flexibility): the channel kinds enumeration is intentionally a TEXT column (not a PostgreSQL enum) so future channel additions (WhatsApp, SMS-to-staff-on-call) land without DDL.

Glossary

Term	Meaning
Staff notification	A message dispatched to one or more BO users about a workflow event (typically a user task requiring action). Contrast with customer notification, which goes to subscribers via DD_NOT-01.
Candidate group	A FOUNDATION_AUTH group code identifying a set of BO users (e.g., <code>kenya-l1-kyc</code> , <code>noc-team</code> , <code>tech-region-NRB-KAREN</code>). Camunda BPMN user tasks declare a <code>camunda:candidateGroups</code> attribute; ICN-01 resolves it to user membership.
Channel	A transport for staff notifications: EMAIL, SLACK, MSTEAMS, IN_APP_PUSH. Each user has a per-channel preference + an effective channel priority.
Template	A named, operator-localised text body with variable placeholders. Identified by a code (e.g., <code>kyc-l1-approval-needed</code>). ICN-01 ships ~30 templates v1.0 KE.
Notification	One ICN-01 dispatch request, fanning out to N recipients × M channels per recipient. Stored as one <code>staff_notification</code> row + N×M <code>staff_notification_delivery</code> rows.
Delivery	One channel-dispatch attempt for one recipient. Has lifecycle: PENDING → DISPATCHED → ACKNOWLEDGED (or FAILED → retried → eventually DISPATCHED-on-fallback / TERMINALLY_FAILED).
Acknowledgement	A staff user marks the notification as seen — either by clicking the deep-link in the message (which calls back to ICN-01) or by opening the user task in the BO UI inbox. Best-effort, not audit-grade.

Term	Meaning
NotificationHandlerResource	A per-module REST handler in FUL-02, WO module (and future modules) that owns the Camunda HTTP connector contract. The handler builds the ICN-01 dispatch request and calls ICN-01's POST /api/staff-notifications. ICN-01 is downstream of these handlers.
Fallback chain	For each recipient, the ordered list of channels to try. Default chain per operator. Per-user overrides supported. Stop at first DISPATCHED (or first ACKNOWLEDGED if mode=AWAIT_ACK).
Suppression	A user's "do not disturb on channel X" state — e.g., user opts out of in-app push during off-hours. Channel-level; not message-level.

2. The dispatch model

2.1 Request flow

```

Camunda BPMN user task fires
↓ HTTP connector (X-Camunda-Connector-Secret)
Per-module NotificationHandlerResource (FUL-02 / WO / future)
↓ Build dispatch request
POST /api/staff-notifications (ICN-01)
↓
ICN-01:
1. Resolve candidateGroup → list of BO users
   via FOUNDATION_AUTH GET /api/groups/{group}/members
2. For each user:
   a. Read user's channel preferences (priority + enabled set)
   b. Render template with templateVariables for each channel
   c. For each enabled channel in priority order:
      - Insert staff_notification_delivery row (PENDING)
      - Async dispatch via channel adapter
      - On success → DISPATCHED + emit Kafka event
      - On fail → FAILED + try next channel
3. Return 202 Accepted with notification_id
↓
Recipient sees the message:
- Email: a link to BO UI deep-link
- Slack/Teams: a card with "Open in BO UI" button
- In-app push: a tray entry that opens the task
↓
Recipient clicks deep-link OR opens task in BO UI:
↓
Either:
- Click on deep-link: GET /api/staff-notifications/{id}/ack?recipient={userId}
- Or: BO UI calls owning-module's task-completion endpoint;
    that module's handler then calls ICN-01 ACK as a side effect
↓
ICN-01 marks staff_notification_delivery row ACKNOWLEDGED + emits event

```

The async dispatch is the **only** workflow path. ICN-01 returns 202 immediately. Workflow waits (or doesn't wait — depends on whether the BPMN's user task has a boundary timer for escalation) for the staff member to actually complete the task in the BO UI.

2.2 Tables overview

ICN-01 owns seven tables:

Table	Purpose
staff_notification_template	Operator-scoped catalog of templates (code, channel variants, subject, body, variables list).
staff_notification_channel_config	Operator-scoped DOMAIN channel configuration only — enabled channels, default priority, fallback mode, retry policy, ACK window. No provider endpoints.
staff_notification_adapter_binding	Operator-scoped MAPPING from (channel kind) → (concrete adapter implementation + adapter-specific config_jsonb). This is where SMTP host, Slack workspace, MS Teams webhook URL, etc. live. Provider-swap = data update here.
staff_notification_user_pref	Per-user DOMAIN preferences only — enabled channels override, priority override, quiet hours, suppression list. No provider identifiers.
staff_notification_user_channel_identity	Per-user, per-channel provider identifier (email address, Slack member ID, MS Teams UPN, etc.) as opaque identity_jsonb. Same decoupling pattern as adapter_binding.

Table	Purpose
staff_notification	One row per dispatch request; references the template + payload.
staff_notification_delivery	One row per (recipient × channel) attempt; lifecycle state.

Plus `staff_notification_inbox_view` (a materialised view providing the "open notifications for this user" inbox). Full DDL in §3.

2.3 Channels

Channel	Transport	Adapter	Reliability
EMAIL	SMTP via internal relay (same as <code>staff_notification_stub v1.0</code> path)	email-adapter	Best, with bounce-back parsing
SLACK	Slack Bot API <code>chat.postMessage</code> (per-operator workspace token in Vault)	slack-adapter	Good; relies on Slack uptime
MSTEAMS	MS Teams Incoming Webhook (per-channel URL in Vault)	msteams-adapter	Good; webhook-based
IN_APP_PUSH	WebSocket fanout to BO UI sessions; fallback to a polled inbox API	inapp-adapter	Conditional on user being logged in

Each adapter implements the same internal Java interface (`ChannelAdapter`) with `dispatch(deliveryRow, renderedMessage)` returning a result + provider-message-id. Adapter implementations are operator-config-driven (which workspace, which webhook, which SMTP relay).

2.4 Templates

A template is identified by (`operator_code`, `template_code`, `channel`) — same template code may have variant rendering per channel (an email is verbose; a Slack card is terse). v1.0 KE seed templates:

- KYC: `kyc-l1-approval-needed`, `kyc-final-approval-needed`, `kyc-rejected-resubmit`
- Install: `install-wo-awaiting-dispatcher`, `install-wo-awaiting-tech-claim`, `install-wo-finalize-confirm-needed`
- Support: `support-wo-tech-needed-internal`, `support-wo-tech-needed-contractor`
- Shifting: `shifting-disconnect-confirm`, `shifting-reconnect-tech-needed`
- Dunning escalation: `dunning-l2-escalation-internal`, `dunning-l3-escalation-internal`
- Finance: `refund-approval-needed`, `adjustment-approval-needed`, `outage-credit-batch-ready`
- NOC: `provisioning-failure-noc-investigate`, `network-incident-confirmation-needed`
- Customer service: `vip-customer-escalation`, `customer-complaint-routed-for-attention`

Templates carry a variable list — `{customerName}`, `{accountNumber}`, `{taskId}`, `{deeplinkUrl}`, `{urgencyLabel}`, etc. The calling module supplies these in `templateVariables`. Variable substitution uses Mustache-style `{{varname}}` rendering. Unknown variables render as `«MISSING:varname»` and emit a warning Kafka event — never a 500.

3. Schemas

3.1 staff_notification_template

```
CREATE TABLE staff_notification_template (
  operator_code      TEXT NOT NULL,
  template_code      TEXT NOT NULL,
  channel            TEXT NOT NULL,
  subject            TEXT NULL,
  body_template       TEXT NOT NULL,
  required_variables TEXT[] NOT NULL DEFAULT '{}',
  optional_variables TEXT[] NOT NULL DEFAULT '{}',
  urgency_default     TEXT NOT NULL DEFAULT 'medium',
  display_name        TEXT NOT NULL,
  description         TEXT NULL,
  enabled             BOOLEAN NOT NULL DEFAULT TRUE,
  -- 'EMAIL' | 'SLACK' | 'MSTEAMS' | 'IN_APP_PUSH'
  -- EMAIL only; NULL for chat channels
  -- Mustache-style {{var}} placeholders
  -- variables the caller must supply
  -- variables the caller may supply
  -- 'low' | 'medium' | 'high'
  -- BO admin UI label
```



```

created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
PRIMARY KEY (operator_code, template_code, channel)
);

```

```
CREATE INDEX idx_template_enabled ON staff_notification_template (operator_code, enabled);
```

Sample data (KE WIK v1.0 seed, abbreviated — full set ~30 templates × 2-4 channel variants each):

operator_code	template_code	channel	subject	body_template (excerpt)	required_variables	urgency_default	
WIK	kyc-l1-approval-needed	EMAIL	[KYC] L1 approval needed: {{customerName}}	Hi {{recipientName}}, Customer {{customerName}} (Order {{orderId}}) is awaiting L1 KYC approval at {{franchiseCode}}. Review: {{deeplinkUrl}} Sophix	{customerName, orderId, franchiseCode, deeplinkUrl}	medium	
WIK	kyc-l1-approval-needed	SLACK	NULL	`id: *KYC L1 approval needed*\n• Customer: {{customerName}}\n• Franchise: {{franchiseCode}}\n• <{{deeplinkUrl}}\`	Open in BO UI>`	{customerName, franchiseCode, deeplinkUrl}	medium
WIK	kyc-l1-approval-needed	IN_APP_PUSH	NULL	KYC L1 needed for {{customerName}} ({{franchiseCode}})	{customerName, franchiseCode}	medium	
WIK	install-wo-awaiting-dispatcher	EMAIL	[WO] Install awaiting tech assignment: {{accountNumber}}	New install WO {{workOrderId}} for {{customerName}} at {{serviceAddress}}. Please assign tech: {{deeplinkUrl}}	{workOrderId, customerName, accountNumber, serviceAddress, deeplinkUrl}	medium	
WIK	install-wo-awaiting-dispatcher	SLACK	NULL	`tools: *Install WO awaiting tech assignment*\n• WO: {{workOrderId}}\n• {{customerName}} @ {{serviceAddress}}\n• <{{deeplinkUrl}}\`	Assign>`	{workOrderId, customerName, serviceAddress, deeplinkUrl}	medium
WIK	refund-approval-needed	EMAIL	[Finance] Refund {{amount}} {{currency}} needs approval	Refund request {{refundRequestId}} for {{customerName}} ({{accountNumber}}): {{amount}} {{currency}}. Reason: {{reasonCode}} Approve: {{deeplinkUrl}}	{refundRequestId, customerName, accountNumber, amount, currency, reasonCode, deeplinkUrl}	high	

operator_code	template_code	channel	subject	body_template (excerpt)	required_variables	urgency_default	
WIK	dunning-l3-escalation-internal	EMAIL	[Dunning L3] Account {{accountNumber}} ({{customerName}}) – overdue {{daysOverdue}}d	Account {{accountNumber}} ({{customerName}}) entered Dunning L3 (suspension).\n\nDaysOverdue: {{daysOverdue}}\n\nAmount: {{amountDue}}\n\nReview: {{deepLinkUrl}}	{accountNumber, customerName, daysOverdue, amountDue, currency, deepLinkUrl}	high	
YASSN	kyc-l1-approval-needed	EMAIL	[KYC] Validation L1 requise: {{customerName}}	Bonjour {{recipientName}},\n\nLe client {{customerName}} (commande {{orderId}}) attend votre validation L1 KYC à {{franchiseCode}}.\n\nLien: {{deepLinkUrl}}\n\n– Sophix	{customerName, orderId, franchiseCode, deepLinkUrl}	medium	
YASSN	kyc-l1-approval-needed	MSTEAMS	NULL	KYC L1 requis \n- Client: {{customerName}}\n- Franchise: {{franchiseCode}}\n- [Ouvrir BO UI]({{deepLinkUrl}})	{customerName, franchiseCode, deepLinkUrl}	medium	

Note YASSN templates are French-language and prefer MS Teams over Slack. KE uses Slack heavily; YASSN uses Teams. Localisation is per-template-per-channel.

3.2 staff_notification_channel_config

Domain configuration only: which channels an operator uses, default priority, fallback semantics, retry policy, ACK window. **Channel-provider endpoint details (SMTP host, Slack workspace, MS Teams webhook URL, push WebSocket URL, etc.) do NOT live here** — they live in the per-channel adapter binding (§3.6), so a channel can swap providers (e.g., move EMAIL from internal SMTP to Microsoft Graph API; move SLACK from Bot API to a different workspace) without altering domain config.

```
CREATE TABLE staff_notification_channel_config (
  operator_code      TEXT PRIMARY KEY,
  enabled_channels   TEXT[] NOT NULL,           -- e.g., ['EMAIL','SLACK','IN_APP_PUSH']
  default_priority   TEXT[] NOT NULL,           -- e.g., ['IN_APP_PUSH','SLACK','EMAIL'] – try in this order
  fallback_mode      TEXT NOT NULL DEFAULT 'PARALLEL', -- 'PARALLEL' | 'SEQUENTIAL_UNTIL_ACK' | 'SEQUENTIAL_UNTIL_DISPATCH'
  retry_max_attempts INTEGER NOT NULL DEFAULT 3,
  retry_backoff_seconds INTEGER[] NOT NULL DEFAULT '{30,120,600}', -- backoff schedule
  ack_window_hours   INTEGER NOT NULL DEFAULT 24, -- after which a non-ACKed notification is auto-archived
  updated_at        TIMESTAMP NOT NULL DEFAULT NOW()
);
```

Sample data:

operator_code	enabled_channels	default_priority	fallback_mode	retry_max_attempts	retry_backoff_seconds	ack_window_hours
WIK	{EMAIL, SLACK, IN_APP_PUSH}	{IN_APP_PUSH, SLACK, EMAIL}	PARALLEL	3	{30,120,600}	24
YASSN	{EMAIL, MSTEAMS}	{MSTEAMS, EMAIL}	PARALLEL	3	{30,120,600}	24
WUG	{EMAIL, SLACK, IN_APP_PUSH}	{IN_APP_PUSH, SLACK, EMAIL}	PARALLEL	3	{30,120,600}	24
WTZ	{EMAIL, IN_APP_PUSH}	{IN_APP_PUSH, EMAIL}	PARALLEL	5	{60,300,900,1800,3600}	48

WTZ uses an extended retry schedule (5 attempts over ~50 minutes) reflecting a less-reliable connectivity profile in the deployment region; the configuration is purely domain-level and doesn't reference any specific provider.

PARALLEL mode: dispatch to all enabled channels simultaneously. First ACK closes all open deliveries for that notification + recipient. Good for high-urgency (don't wait sequentially). SEQUENTIAL_UNTIL_ACK: try priority[0]; if no ACK within `ack_window_hours / N`, try priority[1]; etc. Good for low-volume / high-priority where over-notification is rude. SEQUENTIAL_UNTIL_DISPATCH: try priority[0]; if dispatch fails (channel unavailable), try priority[1]. Stop at first successful dispatch. Good for cost-sensitive (don't double-message).

3.3 staff_notification_user_pref

Per-user DOMAIN preference overrides only — which channels the user wants, in what priority, with what suppression and quiet hours. **Per-channel provider identifiers (email address, Slack user ID, MS Teams UPN, push token, etc.) do NOT live here** — they live in `staff_notification_user_channel_identity` (§3.7), so a new channel's identifier shape lands without altering this table.

```
CREATE TABLE staff_notification_user_pref (
  user_id          TEXT PRIMARY KEY,          -- FOUNDATION_AUTH user id (Keycloak subject)
  operator_code    TEXT NOT NULL,
  enabled_channels TEXT[] NULL,               -- override; NULL = use operator default
  preferred_priority TEXT[] NULL,            -- override; NULL = use operator default
  quiet_hours_start TIME NULL,               -- suppress all but 'high' urgency during these hours
  quiet_hours_end   TIME NULL,               -- pair with start; NULL = no quiet hours
  quiet_hours_timezone TEXT NULL,            -- 'Africa/Nairobi'; defaults to operator TZ
  suppress_channels TEXT[] NOT NULL DEFAULT '{}', -- e.g., ['EMAIL'] = never email this user
  updated_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_by        TEXT NOT NULL            -- 'self' | B0 admin user id
);

CREATE INDEX idx_user_pref_operator ON staff_notification_user_pref (operator_code);
```

Sample data:

user_id	operator_code	enabled_channels	preferred_priority	quiet_hours_start/end (timezone)	suppress_channels
bo_user_csr_07	WIK	NULL	NULL	NULL	{}
bo_user_supervisor_03	WIK	{IN_APP_PUSH, SLACK, EMAIL}	{IN_APP_PUSH, SLACK, EMAIL}	22:00-06:00 (Africa/Nairobi)	{}
team_leader_03	WIK	{EMAIL, SLACK, IN_APP_PUSH}	{EMAIL, IN_APP_PUSH, SLACK}	NULL	{}
bo_user_fraud_team_02	WIK	{EMAIL}	{EMAIL}	NULL	{SLACK, IN_APP_PUSH}
bo_user_yassn_csr_05	YASSN	NULL	NULL	NULL	{}

User `bo_user_supervisor_03` prefers in-app push, with Slack as fallback, then email; has quiet hours 22:00-06:00 local time (Nairobi) during which only high-urgency notifications fire. User `bo_user_fraud_team_02` is email-only — they explicitly suppress Slack and in-app push (privacy / focus preference). Channel-specific provider identifiers (email address, Slack member ID, etc.) for these users live in `staff_notification_user_channel_identity` (§3.7).

3.4 staff_notification

One row per dispatch request from a calling module.

```
CREATE TABLE staff_notification (
  notification_id    TEXT PRIMARY KEY,          -- 'notif_01HZ...' ULID
  operator_code      TEXT NOT NULL,
  source_module      TEXT NOT NULL,             -- 'FUL-02' | 'W0-01' | 'BIL-04' | ...
  source_task_id     TEXT NULL,                 -- Camunda user-task id (if any)
  source_process_instance TEXT NULL,            -- Camunda process instance id (if any)
  source_business_key TEXT NULL,                -- orderId | workOrderId | accountId | refundRequestId
  candidate_group    TEXT NOT NULL,             -- FOUNDATION_AUTH group code
  template_code      TEXT NOT NULL,             -- references staff_notification_template
  template_variables JSONB NOT NULL,            -- variables for template rendering
  urgency            TEXT NOT NULL,             -- 'low' | 'medium' | 'high'
  fallback_mode_override TEXT NULL,             -- override staff_notification_channel_config.fallback_mode
  deeplink_url       TEXT NULL,                -- explicit deeplink to B0 UI; rendered into template
  ack_window_hours   INTEGER NULL,             -- override default ack window
  expected_recipients INTEGER NOT NULL,         -- count of users resolved from candidate_group at creation
  status             TEXT NOT NULL DEFAULT 'PROCESSING', -- 'PROCESSING' | 'DISPATCHED' | 'ACKNOWLEDGED' | 'EXPIRED'
  acknowledged_by    TEXT NULL,                -- the B0 user who first ACKed (clears the notification)
  acknowledged_at    TIMESTAMP NULL,
  created_at         TIMESTAMP NOT NULL DEFAULT NOW(),
);
```

```

    expires_at          TIMESTAMP NULL,                -- computed: created_at + ack_window_hours
    FOREIGN KEY (operator_code, template_code, 'EMAIL') -- example FK to one channel variant; enforce template exists
    REFERENCES staff_notification_template (operator_code, template_code, channel)
);

CREATE INDEX idx_notif_operator      ON staff_notification (operator_code, status);
CREATE INDEX idx_notif_source       ON staff_notification (source_module, source_business_key);
CREATE INDEX idx_notif_candidate_group ON staff_notification (operator_code, candidate_group, status);
CREATE INDEX idx_notif_unack        ON staff_notification (operator_code, status, expires_at) WHERE status NOT IN ('ACKNOWLEDGED', 'DISPATCHED', 'EXPIRED');

```

Sample data:

notification_id	operator_code	source_module	source_task_id	source_business_key	candidate_group	template_code	urgency	expected_recipients	status	acknowledged_by	created_at
notif_01HZ_KYC_L1_JANE	WIK	FUL-02	cam_task_l1_01HZ_JANE	ord_01HZ_JANE_TRIPLE_PLA_Y	kenya-l1-kyc	kyc-l1-approval-needed	medium	4	ACKNOWLEDGED	supervisor_franchise_KE-NRB-04	2026-02-16 17:30:00
notif_01HZ_KYC_FINAL_JANE	WIK	FUL-02	cam_task_final_01HZ_JANE	ord_01HZ_JANE_TRIPLE_PLA_Y	kenya-kyc-leaders	kyc-final-approval-needed	medium	2	ACKNOWLEDGED	team_leader_03	2026-02-17 08:30:00
notif_01HZ_INST_WO_DISP	WIK	WO-01	cam_task_disp_01HZ_KAREN	wo_01HZ_KAREN_INSTALL	tech-region-NRB-KAREN	install-wo-awaiting-dispatcher	medium	3	DISPATCHED	NULL	2026-02-17 14:00:00
notif_01HZ_REFUND_APPROVE	WIK	BIL-05	cam_task_refund_01HZ_X	refund_req_01HZ_X	finance-refunds-approvers	refund-approval-needed	high	2	DISPATCHED	NULL	2026-05-19 11:00:00
notif_01HZ_DUN_L3_PETER	WIK	BIL-04	NULL	acct_01HZ_PETER_KIMANI_NPD_ACCT	kenya-dunning-l3-reviewers	dunning-l3-escalation-internal	high	5	EXPIRED	NULL	2026-04-22 10:00:00

Row 1: Jane's KYC L1 — fanned out to 4 supervisors in kenya- l1- kyc; supervisor_franchise_KE-NRB-04 ACKed first; notification closed.

Row 3: install WO dispatcher notification — fanned to 3 dispatchers in tech-region-NRB-KAREN; still pending dispatcher action; status DISPATCHED (delivered but not ACKed).

Row 5: dunning L3 — no source_task_id (BIL-04 dunning isn't a Camunda user task; it's a Drools-rule-triggered notification); fanned to 5 reviewers; nobody ACKed within the 24h window; auto-EXPIRED.

3.5 staff_notification_delivery

One row per (recipient × channel) dispatch attempt.

```

CREATE TABLE staff_notification_delivery (
    delivery_id          TEXT PRIMARY KEY,                -- 'deliv_01HZ...' ULID
    notification_id      TEXT NOT NULL REFERENCES staff_notification(notification_id),
    recipient_user_id    TEXT NOT NULL,                  -- FOUNDATION_AUTH user id
    channel              TEXT NOT NULL,                  -- 'EMAIL' | 'SLACK' | 'MSTEAMS' | 'IN_APP_PUSH'
    channel_priority_idx INTEGER NOT NULL,                -- 0-based index in the effective priority order
    status              TEXT NOT NULL DEFAULT 'PENDING', -- 'PENDING' | 'DISPATCHED' | 'ACKNOWLEDGED' | 'FAILED' | 'TERMINATED'
    attempts            INTEGER NOT NULL DEFAULT 0,
    last_attempt_at     TIMESTAMP NULL,
    provider_message_id TEXT NULL,                        -- e.g., Slack ts, email Message-ID
    provider_response    JSONB NULL,                     -- raw provider response (sanitized)
    failure_reason       TEXT NULL,                      -- 'AUTH_ERROR' | 'CHANNEL_DOWN' | 'TOKEN_EXPIRED' | etc.
    acknowledged_at     TIMESTAMP NULL,
    created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
    next_retry_at       TIMESTAMP NULL                    -- for FAILED → PENDING re-try scheduling
);

CREATE INDEX idx_delivery_notification ON staff_notification_delivery (notification_id);
CREATE INDEX idx_delivery_recipient   ON staff_notification_delivery (recipient_user_id, status);
CREATE INDEX idx_delivery_retry_due   ON staff_notification_delivery (next_retry_at) WHERE status = 'FAILED' AND next_retry_at < NOW();
CREATE UNIQUE INDEX uq_delivery_notif_recipient_channel
    ON staff_notification_delivery (notification_id, recipient_user_id, channel);

```

Sample data for notif_01HZ_KYC_L1_JANE (4 recipients × 2-3 channels each):

delivery_id	recipient_user_id	channel	status	attempts	provider_message_id	acknowledged_at
deliv_01HZ_J_KE_N04_PUSH	supervisor_franchise_KE-NRB-04	IN_APP_PUSH	ACKNOWLEDGED	1	push_msg_01HZ_X1	2026-02-17 08:15:00
deliv_01HZ_J_KE_N04_SLACK	supervisor_franchise_KE-NRB-04	SLACK	DISPATCHED	1	slack_msg_1707..._X	NULL
deliv_01HZ_J_KE_N04_EMAIL	supervisor_franchise_KE-NRB-04	EMAIL	DISPATCHED	1	<msg-01HZ-1234@wik.sophix>	NULL
deliv_01HZ_J_KE_N05_PUSH	supervisor_franchise_KE-NRB-05	IN_APP_PUSH	SUPPRESSED	0	NULL	NULL
deliv_01HZ_J_KE_N05_SLACK	supervisor_franchise_KE-NRB-05	SLACK	FAILED	3	NULL	NULL
deliv_01HZ_J_KE_N05_EMAIL	supervisor_franchise_KE-NRB-05	EMAIL	DISPATCHED	1	<msg-01HZ-1235@wik.sophix>	NULL

Recipient supervisor_franchise_KE-NRB-04 ACKed via in-app push (was logged into BO UI); Slack + email rows stay DISPATCHED but the parent staff_notification is now ACKNOWLEDGED so further actions on those deliveries are no-ops.

Recipient supervisor_franchise_KE-NRB-05 was on quiet-hours for in-app push (SUPPRESSED); Slack failed 3 times (rate limit?); email succeeded.

3.6 staff_notification_adapter_binding

Channel-provider endpoint configuration. **This is where SMTP host, Slack workspace ID, MS Teams webhook URL, Microsoft Graph tenant ID, AWS SES region, Mailgun API base, push WebSocket URL, etc. live.** ICN-01 reads this table to know which adapter implementation handles each (operator, channel) combination and how to configure it.

Each binding row identifies one adapter implementation (e.g., smtp-classic, microsoft-graph-mail, slack-bot-api, msteams-incoming-webhook, msteams-graph-channel, inapp-websocket-fanout, aws-ses, etc.) and carries an opaque config_jsonb whose shape is interpreted only by that adapter. Secret material (passwords, API tokens, OAuth refresh tokens) is NEVER stored inline in config_jsonb — only Vault path references.

The decoupling means:

- KE may use smtp-classic for EMAIL today and migrate to microsoft-graph-mail tomorrow by inserting a new binding row + removing the old; no schema change, no code change, no domain-config change.
- A future operator can plug in a wholly new adapter implementation (twilio-programmable-chat, whatsapp-business-api, pagerduty-events-v2) by deploying the adapter code and inserting the binding row.
- Multiple operators can share an adapter implementation with completely different config (e.g., both KE and WUG use slack-bot-api but each has its own workspace + token).

```
CREATE TABLE staff_notification_adapter_binding (
  operator_code      TEXT NOT NULL,
  channel            TEXT NOT NULL,
  adapter_impl       TEXT NOT NULL,
  config_jsonb       JSONB NOT NULL,
  enabled            BOOLEAN NOT NULL DEFAULT TRUE,
  notes              TEXT NULL,
  created_at         TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at         TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_by         TEXT NOT NULL,
  PRIMARY KEY (operator_code, channel)
);

CREATE INDEX idx_binding_enabled ON staff_notification_adapter_binding (operator_code, enabled);
```

Exactly one binding per (operator_code, channel) — that's the active adapter for that channel for that operator. To swap providers: update the row (or insert new + delete old in a transaction).

Sample data:

operator_code	channel	adapter_impl	config_jsonb (excerpt)	enabled
WIK	EMAIL	smtp-classic	{"host": "smtp-relay-internal.wik.sophix.local", "port": 587, "starttls": true, "from": "sophix-noreply@wik.sophix.local", "auth_vault_ref": "secret/icn/wik/smtp"}	true

operator_code	channel	adapter_impl	config_jsonb (excerpt)	enabled
WIK	SLACK	slack-bot-api	{"workspace_id": "T7K1S0PH1XKE", "bot_token_vault_ref": "secret/icn/wik/slack/bot-token", "base_url": "https://slack.com/api"}	true
WIK	IN_APP_PUSH	inapp-websocket-fanout	{"ws_endpoint": "wss://push.wik.sophix.com/staff", "polling_fallback_endpoint": "https://push.wik.sophix.com/staff/inbox", "session_jwt_audience": "icn-01"}	true
YASSN	EMAIL	microsoft-graph-mail	{"tenant_id": "yassn-tenant-uuid", "app_id": "yassn-graph-app-id", "auth_vault_ref": "secret/icn/yassn/graph", "from": "sophix-noreply@yassn.sophix.local"}	true
YASSN	MSTEAMS	msteams-incoming-webhook	{"channel_webhooks": {"yassn-kyc-reviews": "https://outlook.office.com/webhook/...", "yassn-finance-approvals": "https://outlook.office.com/webhook/...", "yassn-noc": "https://outlook.office.com/webhook/..."}, "vault_ref": "secret/icn/yassn/msteams-webhooks"}	true
WUG	EMAIL	smtp-classic	{"host": "smtp-relay-internal.wug.sophix.local", "port": 587, "starttls": true, "from": "sophix-noreply@wug.sophix.local", "auth_vault_ref": "secret/icn/wug/smtp"}	true
WUG	SLACK	slack-bot-api	{"workspace_id": "T7K1S0PH1XUG", "bot_token_vault_ref": "secret/icn/wug/slack/bot-token", "base_url": "https://slack.com/api"}	true
WUG	IN_APP_PUSH	inapp-websocket-fanout	{"ws_endpoint": "wss://push.wug.sophix.com/staff", "polling_fallback_endpoint": "https://push.wug.sophix.com/staff/inbox", "session_jwt_audience": "icn-01"}	true
WTZ	EMAIL	smtp-classic	{"host": "smtp-relay-internal.wtz.sophix.local", "port": 587, "starttls": true, "from": "sophix-noreply@wtz.sophix.local", "auth_vault_ref": "secret/icn/wtz/smtp"}	true
WTZ	IN_APP_PUSH	inapp-websocket-fanout	{"ws_endpoint": "wss://push.wtz.sophix.com/staff", "polling_fallback_endpoint": "https://push.wtz.sophix.com/staff/inbox", "session_jwt_audience": "icn-01"}	true

Note: WIK uses three channels (EMAIL, SLACK, IN_APP_PUSH) — three binding rows. YASSN uses two channels (EMAIL via Microsoft Graph API rather than SMTP, MSTEAMS via Incoming Webhooks) — two binding rows. The channel_webhooks map in YASSN's MSTEAMS config lets the adapter pick the right Teams channel based on the candidate_group of the notification (configured per-operator outside this row, in the adapter's deployment config — see §7).

A second example showing provider migration in progress on KE EMAIL: imagine WIK is mid-migration from smtp-classic to microsoft-graph-mail. The transition is a single row update + a code deploy with the new adapter implementation. During testing, operators can briefly run with enabled=false on the new binding while keeping the old one active; once cutover, swap them. No domain-config impact.

3.7 staff_notification_user_channel_identity

Per-user, per-channel provider identifier. **This is where a user's email address, Slack member ID, MS Teams UPN, push token, etc. live** — each as an opaque identity_jsonb whose shape is interpreted by the corresponding channel adapter. Mirrors the staff_notification_adapter_binding pattern: domain (user_pref) is decoupled from provider-specifics (channel identity).

When the binding swaps adapter implementation, the identity shape may change too (e.g., slack-bot-api uses {"slack_user_id":"U07..."}; a hypothetical slack-connect-api might use {"slack_external_id":"E123..."}). Migration of identities is a data-only update on this table; no schema impact.

```
CREATE TABLE staff_notification_user_channel_identity (
  user_id          TEXT NOT NULL,          -- FOUNDATION_AUTH user id (Keycloak subject)
  channel          TEXT NOT NULL,          -- 'EMAIL' | 'SLACK' | 'MSTEAMS' | 'IN_APP_PUSH'
  operator_code    TEXT NOT NULL,          -- denormalized for cross-tenant queries
  identity_jsonb   JSONB NOT NULL,         -- shape is adapter-specific; opaque to ICN-01
  source           TEXT NOT NULL,          -- 'self' | 'admin' | 'auto-resolved-from-auth' | 'directory-sync'
  verified         BOOLEAN NOT NULL DEFAULT FALSE,
  verified_at      TIMESTAMP NULL,
  updated_at       TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_by       TEXT NOT NULL,
  PRIMARY KEY (user_id, channel)
);

CREATE INDEX idx_uci_operator ON staff_notification_user_channel_identity (operator_code);
CREATE INDEX idx_uci_channel ON staff_notification_user_channel_identity (channel);
```

Sample data:

user_id	channel	identity_jsonb	source	verified
bo_user_csr_07	EMAIL	{"address":"csr07@wik.wananchi.com"}	auto-resolved-from-auth	true
bo_user_csr_07	SLACK	{"slack_user_id":"U07K1S0PH1XKE"}	directory-sync	true
bo_user_supervisor_03	EMAIL	{"address":"sup03@wik.wananchi.com"}	auto-resolved-from-auth	true
bo_user_supervisor_03	SLACK	{"slack_user_id":"U03K1S0PH1XKE"}	directory-sync	true
bo_user_supervisor_03	IN_APP_PUSH	{}	auto-resolved-from-auth	true
team_leader_03	EMAIL	{"address":"tl03@wik.wananchi.com"}	auto-resolved-from-auth	true
team_leader_03	SLACK	{"slack_user_id":"U03TL1KE"}	directory-sync	true
bo_user_fraud_team_02	EMAIL	{"address":"fraud02@wik.wananchi.com"}	auto-resolved-from-auth	true
bo_user_yassn_csr_05	EMAIL	{"address":"csr05@yassn.wananchi.com"}	auto-resolved-from-auth	true
bo_user_yassn_csr_05	MSTEAMS	{"upn":"csr05@yassn.wananchi.com","object_id":"d3bd8eb6-..."}	directory-sync	true
bo_user_yassn_l1_kyc_02	MSTEAMS	{"upn":"l1kyc02@yassn.wananchi.com","object_id":"e4af9c12-..."}	directory-sync	true

Note row 5 for bo_user_supervisor_03 IN_APP_PUSH: empty identity_jsonb is valid — the inapp-websocket-fanout adapter uses user_id directly to look up active WebSocket sessions, so no extra identifier is needed; the row's existence just confirms "this user is set up for IN_APP_PUSH." If the row didn't exist, the adapter would still find live sessions via user_id, so this row is informational for some channels (see §7.0 resolution rules).

Note row 10 YASSN MSTEAMS includes BOTH upn and object_id — msteams-incoming-webhook adapter doesn't use the user identity (delivery is channel-scoped, not user-scoped, per §7.3); but if YASSN migrates to msteams-graph-channel later, that adapter will need the user identifier to post to user DMs, and the object_id is the format Microsoft Graph prefers. Capturing both now means the future migration is data-only.

3.7.1 Resolution semantics — when there's no row

A user may not have a row for every channel they could potentially receive notifications on. ICN-01's resolution policy at dispatch time:

1. If a row exists for (user_id, channel) → pass identity_jsonb to the adapter.
2. If no row exists → pass null to the adapter. The adapter then decides its fallback:
 - smtp-classic and microsoft-graph-mail: fall back to FOUNDATION_AUTH GET /api/users/{user_id} to read the email attribute from the user's Keycloak profile; if found, proceed with that address and OPTIONALLY backfill the identity row (the adapter MAY insert a new identity row with source='auto-resolved-from-auth', verified=false — convenience, not required).
 - slack-bot-api: fall back to FOUNDATION_AUTH email lookup, then Slack users.lookupByEmail against the workspace; on success, MAY backfill the identity row.
 - msteams-incoming-webhook: NO fallback — delivery is channel-scoped, not user-scoped (the webhook posts to a Teams channel based on candidate_group_to_channel mapping in the binding's config, not to a user). The adapter ignores the per-user identity entirely. The identity row, if present, is just for forward-compatibility.
 - msteams-graph-channel (future): fall back to FOUNDATION_AUTH email lookup → MS Graph users lookup by mail.
 - inapp-websocket-fanout: NO fallback needed — adapter uses user_id to look up active WebSocket sessions. The identity row's existence is informational but not required.

This policy means: a user can receive notifications on most channels without any identity row pre-populated, provided FOUNDATION_AUTH carries their email. The identity table is for performance (skip the AUTH call) + for cases where the channel identifier is NOT derivable from email (e.g., a Slack workspace where users joined with a different email than their Keycloak account uses).

3.7.2 Directory sync

A scheduled job (identity-sync-<operator>-v<n>, daily) reconciles staff_notification_user_channel_identity with each operator's external directories:

- Pulls every staff user from FOUNDATION_AUTH for the operator
- For each user × each enabled channel, calls the adapter's resolveIdentity(userProfile) method (a separate ChannelAdapter interface method beyond dispatch)
- Upserts the resulting identity_jsonb with source='directory-sync', verified=true (since it came from the authoritative directory)
- Removes rows for users who've been deactivated in FOUNDATION_AUTH

Users can override sync-managed identities via the BO UI (source='self'); self-managed rows are not overwritten by sync. Admin overrides (source='admin') are honored similarly.

4. Rules

Rule	Description
R-ICN-01-D-1	A notification is ACKNOWLEDGED the moment ANY one of its recipients ACKs ANY one of their deliveries. The staff_notification.acknowledged_by records the first ACKer. Subsequent ACKs from other recipients are recorded on their respective delivery rows but do not change the parent.
R-ICN-01-D-2	An ACKNOWLEDGED notification ceases all retry activity. In-flight deliveries that haven't been dispatched yet move to status SUPPRESSED (we don't pester other staff once the task is in flight).
R-ICN-01-D-3	A notification with no ACK by expires_at (= created_at + ack_window_hours) moves to status EXPIRED. EXPIRED notifications stay in the table for audit; their deliveries are not retried. The calling workflow MUST have its own escalation timer (Camunda boundary timer) — ICN-01 does NOT escalate.

Rule	Description
R-ICN-01-D-4	Channel selection per recipient: (1) take <code>staff_notification_user_pref.preferred_priority</code> if set, else <code>staff_notification_channel_config.default_priority</code> ; (2) intersect with <code>staff_notification_user_pref.enabled_channels</code> (or operator default); (3) subtract <code>staff_notification_user_pref.suppress_channels</code> ; (4) apply quiet-hours rule (R-ICN-01-D-5); the remainder is the recipient's effective channel list, in priority order.
R-ICN-01-D-5	Quiet hours suppress all channels EXCEPT for <code>urgency='high'</code> notifications, which dispatch regardless. Quiet hours are evaluated against <code>staff_notification_user_pref.quiet_hours_timezone</code> (defaults to the operator's TZ).
R-ICN-01-D-6	<code>fallback_mode = PARALLEL</code> : all enabled channels for a recipient receive simultaneous dispatch attempts. <code>SEQUENTIAL_UNTIL_ACK</code> : attempt <code>priority[0]</code> ; if no ACK within <code>ack_window_hours / channel_count</code> , fall to <code>priority[1]</code> , etc. <code>SEQUENTIAL_UNTIL_DISPATCH</code> : attempt <code>priority[0]</code> ; if FAILED, fall to <code>priority[1]</code> , etc. — stop at first DISPATCHED.
R-ICN-01-D-7	A FAILED delivery is retried per <code>staff_notification_channel_config.retry_backoff_seconds</code> (default {30, 120, 600}). After <code>retry_max_attempts</code> failures (default 3), status moves to <code>TERMINALLY_FAILED</code> and the next channel in the priority list is attempted (for <code>SEQUENTIAL_*</code> modes) or no further action (for <code>PARALLEL</code>).
R-ICN-01-D-8	If ALL recipients have ALL channels <code>TERMINALLY_FAILED</code> or <code>SUPPRESSED</code> , the parent <code>staff_notification.status</code> moves to <code>EXPIRED</code> with reason <code>ALL_CHANNELS_EXHAUSTED</code> . A high-severity Kafka event <code>StaffNotificationFailedToReachAnyone</code> fires for ops alerting.
R-ICN-01-D-9	The dispatch is asynchronous; <code>POST /api/staff-notifications</code> returns 202 the moment the parent row + recipient delivery rows are persisted. Actual channel adapter calls run in a background worker pool. This means a Camunda HTTP connector firing into ICN-01 gets a 202 back quickly even if Slack is slow.
R-ICN-01-D-10	ICN-01 NEVER dispatches to a customer. The schema does not contain any FK to customer or <code>customer_account</code> . Calling modules may include customer identifiers in <code>template_variables</code> for display in the rendered message (e.g., <code>{{customerName}}</code>), but the recipients of the message are STRICTLY staff users resolved from <code>candidate_group</code> via <code>FOUNDATION_AUTH</code> .
R-ICN-01-D-11	Idempotency: <code>POST /api/staff-notifications</code> accepts an optional Idempotency-Key header. Repeated calls with the same key within 24h return the existing <code>notification_id</code> with HTTP 200 (not 202), and do NOT re-dispatch. This protects against Camunda HTTP connector retries during transient ICN-01 unavailability.
R-ICN-01-D-12	All adapter-specific secret material (SMTP credentials, Slack bot tokens, MS Teams webhook URLs, Microsoft Graph app secrets, etc.) is stored ONLY as Vault path references inside <code>staff_notification_adapter_binding.config_jsonb</code> . Never inline values. Adapters resolve the Vault reference at dispatch time (or at adapter init, with TTL cache).
R-ICN-01-D-13	Template rendering treats missing required variables as a soft-error: render as <code>«MISSING:varname»</code> and emit <code>StaffNotificationTemplateRenderWarning</code> . Calling modules can subscribe to this Kafka topic to detect their own template-variable bugs. ICN-01 never returns 500 for a missing variable.
R-ICN-01-D-14	PII minimization: templates MUST NOT reference customer identification numbers (<code>identification_number_1</code> , <code>identification_number_2</code>), full address, or any flag value (<code>NPD</code> , <code>FRAUD_SUSPECTED</code>). Customer name + account number are the maximum PII surface for staff notifications. Enforced by template-content review at registration time; not by code.
R-ICN-01-D-15	Per-operator templates: a template with (<code>operator_code='WIK'</code> , <code>template_code='kyc-l1-approval-needed'</code> , <code>channel='SLACK'</code>) is independent from (<code>operator_code='YASSN'</code> , <code>template_code='kyc-l1-approval-needed'</code> , <code>channel='MSTEAMS'</code>). The calling module passes only <code>template_code + operator_code</code> ; ICN-01 picks the right channel variant per recipient's preferred channel.
R-ICN-01-D-16	Workflow modules NEVER directly insert into <code>staff_notification_delivery</code> — only via the <code>POST /api/staff-notifications</code> API. This is the single ingress point.

Rule	Description
R-ICN-01-D-17	Adapter binding resolution: when dispatching a delivery row for (operator_code, channel), ICN-01 reads staff_notification_adapter_binding to get the active adapter_impl + config_jsonb. The adapter_impl value MUST match a registered Java adapter class implementing ChannelAdapter (§7); unknown adapter_impl causes the delivery to TERMINALLY_FAIL with failure_reason = ADAPTER_NOT_REGISTERED and emits an ops alert. The binding's config_jsonb is passed verbatim to the adapter — ICN-01 does NOT validate its shape; that's the adapter's responsibility at init.
R-ICN-01-D-18	Disabled bindings (enabled=false) cause deliveries for that (operator_code, channel) to be marked SUPPRESSED at dispatch time with failure_reason = ADAPTER_BINDING_DISABLED. The notification's other channels (if any) still attempt dispatch normally. This supports temporary channel takedowns for maintenance without touching domain config.
R-ICN-01-D-19	Per-user channel-identity resolution: when dispatching to recipient user_id on channel C, ICN-01 reads staff_notification_user_channel_identity for (user_id, C). If a row exists, identity_jsonb is passed verbatim to the adapter (opaque to ICN-01). If no row exists, identity_jsonb=null is passed and the adapter applies its own fallback resolution (typically FOUNDATION_AUTH user-profile lookup; per-adapter behavior in §7.0). Adapter-level resolution failure → delivery TERMINALLY_FAILED with failure_reason = IDENTITY_UNRESOLVABLE.

5. REST API

All endpoints scoped to the requesting actor's operator via auth context. Cross-operator calls reserved for sophix-superadmin.

5.1 Notification dispatch

```
POST /api/staff-notifications HTTP/1.1
Authorization: Bearer <module-service-account-token>
Idempotency-Key: ful-02-kyc-l1-cam_task_l1_01HZ_JANE
Content-Type: application/json

{
  "operatorCode":      "WIK",
  "sourceModule":      "FUL-02",
  "sourceTaskId":      "cam_task_l1_01HZ_JANE",
  "sourceProcessInstance": "01HZ_FUL02_INSTANCE_JANE",
  "sourceBusinessKey": "ord_01HX_JANE_TRIPLE_PLAY",
  "candidateGroup":    "kenya-l1-kyc",
  "templateCode":      "kyc-l1-approval-needed",
  "templateVariables": {
    "customerName": "Jane Mwangi",
    "orderId":      "ord_01HX_JANE_TRIPLE_PLAY",
    "franchiseCode": "KE-NRB-FRANCHISE-04",
    "deeplinkUrl":  "https://bo.wik.sophix.com/tasks/cam_task_l1_01HZ_JANE"
  },
  "urgency":           "medium",
  "ackWindowHours":    24
}

→ 202 Accepted
{
  "notificationId":      "notif_01HZ_KYC_L1_JANE",
  "expectedRecipients": 4,
  "createdAt":           "2026-02-16T17:30:00Z",
  "expiresAt":           "2026-02-17T17:30:00Z"
}
```

If Idempotency-Key matches a prior request within 24h:

```
→ 200 OK
{
  "notificationId":      "notif_01HZ_KYC_L1_JANE",
  "expectedRecipients": 4,
  "createdAt":           "2026-02-16T17:30:00Z",
  "expiresAt":           "2026-02-17T17:30:00Z",
  "idempotentReplay":    true
}
```

Errors:

Status	When
400 MISSING_REQUIRED_VARIABLES	templateVariables lacks a variable listed in the template's required_variables.
404 TEMPLATE_NOT_FOUND	(operatorCode, templateCode) doesn't resolve to an enabled template.
404 CANDIDATE_GROUP_EMPTY	FOUNDATION_AUTH returned 0 members for candidateGroup. The notification IS still created (audit trail), but status = EXPIRED immediately with reason NO_RECIPIENTS.
422 OPERATOR_MISMATCH	The calling actor's operator differs from operatorCode and actor lacks sophix-superadmin role.
429 RATE_LIMITED	Per-source-module rate limit hit (default 1,000 notifications per minute per module).

5.2 Acknowledgement (called by deep-link click or by BO UI)

POST /api/staff-notifications/{notification_id}/ack HTTP/1.1
Authorization: Bearer <staff-user-token>

```
{
  "recipientUserId": "supervisor_franchise_KE-NRB-04",
  "ackChannel":      "IN_APP_PUSH",
  "ackContext":      {
    "userAgent":      "BO-UI/2.4.1",
    "viewSource":      "deeplink"
  }
}

→ 200 OK
{
  "notificationId": "notif_01HZ_KYC_L1_JANE",
  "status":         "ACKNOWLEDGED",
  "acknowledgedBy": "supervisor_franchise_KE-NRB-04",
  "acknowledgedAt": "2026-02-17T08:15:00Z"
}
```

recipientUserId MUST be in the JWT's sub claim (a staff user can only ACK on their own behalf — no proxy-ACK). Cross-user ACK returns 403 NOT_RECIPIENT.

A second ACK for an already-acknowledged notification returns 200 with idempotentAck: true — no error, no state change.

5.3 Inbox query (BO UI)

GET /api/staff-notifications/inbox?status=DISPATCHED&limit=20 HTTP/1.1
Authorization: Bearer <staff-user-token>

```
→ 200 OK
{
  "notifications": [
    {
      "notificationId": "notif_01HZ_INST_WO_DISP",
      "sourceModule":   "WO-01",
      "sourceBusinessKey": "wo_01HZ_KAREN_INSTALL",
      "templateCode":    "install-wo-awaiting-dispatcher",
      "renderedBody":    "...",
      "urgency":         "medium",
      "deeplinkUrl":     "https://bo.wik.sophix.com/work-orders/wo_01HZ_KAREN_INSTALL",
      "deliveryStatus":  "DISPATCHED",
      "deliveredAt":     "2026-02-17T14:00:32Z",
      "expiresAt":       "2026-02-18T14:00:32Z"
    },
    {...}
  ],
  "totalUnread":       12
}
```

Inbox filters: status (PENDING/DISPATCHED/ACKNOWLEDGED), sourceModule, urgency, since (timestamp). The BO UI polls this every 30s as a fallback for missed WebSocket pushes.

5.4 Catalog management

Endpoint	Purpose
GET /api/staff-notification-templates?operatorCode=WIK	List templates for an operator.
POST /api/staff-notification-templates	Create or update a template (BO admin only).

Endpoint	Purpose
PUT /api/staff-notification-templates/{operator}/{code}/{channel}/disable	Disable a template (does not delete).
GET /api/staff-notification-channel-config?operatorCode=WIK	Read operator channel config (domain config only — no provider endpoints).
PUT /api/staff-notification-channel-config/{operatorCode}	Update operator channel config (BO admin only; requires sophix-superadmin or <operator>-icn-admin role).
GET /api/staff-notification-adapter-bindings?operatorCode=WIK	List active adapter bindings for an operator (returns rows with config_jsonb redacted of any Vault refs for safe BO UI display).
GET /api/staff-notification-adapter-bindings/{operator}/{channel}	Read a specific binding (full config_jsonb returned to authorised <operator>-icn-admin only; Vault refs visible but secrets never resolved here).
PUT /api/staff-notification-adapter-bindings/{operator}/{channel}	Create or update a binding (requires <operator>-icn-admin or sophix-superadmin). Triggers an adapter re-init for that (operator, channel).
POST /api/staff-notification-adapter-bindings/{operator}/{channel}/test	Dry-run dispatch a test notification through the binding to a designated test recipient (configured per binding). Used during provider migration to verify the new adapter works before flipping enabled=true.
GET /api/staff-notification-user-pref/{user_id}	Read user's preference overrides.
PUT /api/staff-notification-user-pref/{user_id}	Update user's preference overrides. Staff can update their own; supervisors can update reports'.
GET /api/staff-notification-user-channel-identity/{user_id}	List all channel identity rows for a user (all channels).
GET /api/staff-notification-user-channel-identity/{user_id}/{channel}	Read identity row for a specific (user, channel).
PUT /api/staff-notification-user-channel-identity/{user_id}/{channel}	Create or update an identity row. Staff can self-manage their own; supervisors / <operator>-icn-admin can manage on behalf of reports. Sets source='self' or 'admin' accordingly.
DELETE /api/staff-notification-user-channel-identity/{user_id}/{channel}	Remove an identity row. Subsequent dispatches fall back to adapter-level resolution (per §7.0 step 6).
GET /api/icn/adapter-registry	List the adapter implementations registered in this deployment (e.g., [smtp-classic, microsoft-graph-mail, slack-bot-api, msteams-incoming-webhook, inapp-websocket-fanout]). Used by BO admin UI to populate the adapter_impl dropdown when editing bindings.

5.5 Observability

Endpoint	Purpose
GET /api/staff-notifications/{id}	Inspect a specific notification including all delivery rows (for ops debugging).
GET /api/staff-notifications?source_module=BIL-04&since=2026-05-01	Query by source for audit / triage.
GET /api/staff-notifications/stats?operatorCode=WIK&since=2026-05-01	Aggregate stats: ack rate, channel success rate, mean time to ACK. Used by ops dashboards.

6. Kafka events

ICN-01 publishes on `sophix.icn.*`. No events are consumed inbound (ICN-01 is a downstream service; calling modules call via REST, not Kafka).

6.1 Events published

Event type	Topic	Emitted when	Consumers
StaffNotificationCreated	<code>sophix.icn.notification-created</code>	New <code>staff_notification</code> row	Ops dashboards, CVM analytics
StaffNotificationDelivered	<code>sophix.icn.notification-delivered</code>	Any delivery row reaches DISPATCHED	Ops
StaffNotificationAcknowledged	<code>sophix.icn.notification-acknowledged</code>	Notification ACKed by any recipient	Workflow modules (informational; not for state-change), CVM
StaffNotificationDeliveryFailed	<code>sophix.icn.delivery-failed</code>	A delivery transitions to TERMINALLY_FAILED	Ops alerting

Event type	Topic	Emitted when	Consumers
StaffNotificationFailedToReachAnyone	sophix.icn.failed-no-recipient	All recipient/channel combinations exhausted (R-ICN-01-D-8)	High-severity ops alerting
StaffNotificationExpired	sophix.icn.notification-expired	Notification expired without ACK	Ops + calling-module observability
StaffNotificationTemplateRenderWarning	sophix.icn.template-render-warning	Template rendered with «MISSING: varname»	Source-module dev teams (their template-variable bug)

6.2 Sample event — StaffNotificationAcknowledged

```
{
  "eventType": "StaffNotificationAcknowledged",
  "aggregateId": "notif_01HZ_KYC_L1_JANE",
  "timestamp": "2026-02-17T08:15:00.234Z",
  "payload": {
    "notificationId": "notif_01HZ_KYC_L1_JANE",
    "operatorCode": "WIK",
    "sourceModule": "FUL-02",
    "sourceTaskId": "cam_task_l1_01HZ_JANE",
    "sourceBusinessKey": "ord_01HX_JANE_TRIPLE_PLAY",
    "candidateGroup": "kenya-l1-kyc",
    "templateCode": "kyc-l1-approval-needed",
    "urgency": "medium",
    "acknowledgedBy": "supervisor_franchise_KE-NRB-04",
    "ackChannel": "IN_APP_PUSH",
    "createdAt": "2026-02-16T17:30:00Z",
    "acknowledgedAt": "2026-02-17T08:15:00Z",
    "timeToAckSeconds": 52500,
    "expectedRecipients": 4,
    "dispatchedToRecipients": 4,
    "dispatchedToChannels": 10
  }
}
```

6.3 Sample event — StaffNotificationFailedToReachAnyone

```
{
  "eventType": "StaffNotificationFailedToReachAnyone",
  "aggregateId": "notif_01HZ_NOC_OUTAGE_X",
  "timestamp": "2026-05-20T14:23:00.001Z",
  "payload": {
    "notificationId": "notif_01HZ_NOC_OUTAGE_X",
    "operatorCode": "WIK",
    "sourceModule": "NMS-MONITORING",
    "candidateGroup": "noc-team",
    "templateCode": "noc-critical-outage",
    "urgency": "high",
    "expectedRecipients": 8,
    "deliveryAttempts": 32,
    "failureBreakdown": {
      "EMAIL_TERMINALLY_FAILED": 8,
      "SLACK_TERMINALLY_FAILED": 8,
      "IN_APP_PUSH_SUPPRESSED": 8
    },
    "rootCauseHint": "SMTP relay smtp-relay-internal.wik.sophix.local unreachable; Slack workspace T7K1S0PH1XKE returning",
    "createdAt": "2026-05-20T14:13:00Z",
    "expiredAt": "2026-05-20T14:23:00Z"
  }
}
```

This is a **page-the-ops-team event**. It means ICN-01 tried everything and reached no one — typically an infrastructure outage. Ops dashboards should surface this prominently; on-call should investigate within minutes.

7. Channel adapters

A **channel** is a logical kind (EMAIL, SLACK, MSTEAMS, IN_APP_PUSH). An **adapter** is a concrete implementation that talks to one specific provider for that channel kind. For each (operator_code, channel) exactly one adapter implementation is active at any time — selected by staff_notification_adapter_binding.adapter_impl (§3.6).

Every adapter implements the same Java interface:

```
interface ChannelAdapter {
  String adapterImplCode(); // matches staff_notification_adapter_binding.adapter_impl

  void init(JsonNode configJsonb, SecretResolver secretResolver);
}
```

```

// configJsonb is the adapter binding's config_jsonb passed verbatim;
// secretResolver resolves Vault path refs found inside it.

ChannelDispatchResult dispatch(
    StaffNotificationDelivery row,
    RenderedMessage message,
    RecipientInfo recipient
);

// Optional – called by directory-sync job to populate staff_notification_user_channel_identity
default Optional<JsonNode> resolveIdentity(UserProfile userProfile) {
    return Optional.empty();
}
}

record RecipientInfo(
    String    userId,                // FOUNDATION_AUTH user id
    JsonNode identityJsonb,          // from staff_notification_user_channel_identity, or null
    Supplier<UserProfile> fallbackProfileLookup // lazily calls FOUNDATION_AUTH if needed
);

record ChannelDispatchResult(
    String providerMessageId, // e.g., Slack 'ts', email Message-ID
    boolean success,
    String failureReason,     // when !success
    Map<String, Object> providerResponse
);

```

Adapter implementations are discovered at boot via Java's `ServiceLoader` (META-INF/services/com.sophix.icn.ChannelAdapter). Adding a new adapter is: write the class, register it via `ServiceLoader`, deploy the jar, insert a binding row.

7.0 Dispatch resolution — how an adapter gets picked and called

For every PENDING `staff_notification_delivery` row, the ICN-01 dispatch worker performs these steps before calling an adapter:

- 1. Read the delivery row.** From the row: `notification_id`, `recipient_user_id`, `channel`. From the parent `staff_notification`: `operator_code`, `template_code`, `template_variables`.
- 2. Resolve the adapter binding.** Query `staff_notification_adapter_binding` for (`operator_code`, `channel`):

```

SELECT adapter_impl, config_jsonb, enabled
FROM staff_notification_adapter_binding
WHERE operator_code = $1 AND channel = $2;

```

If no row → delivery `TERMINALLY_FAILED` with `failure_reason = NO_BINDING_FOR_CHANNEL`. If `enabled = false` → delivery `SUPPRESSED` with `failure_reason = ADAPTER_BINDING_DISABLED` (per R-ICN-01-D-18).

- 3. Look up the registered ChannelAdapter instance.** ICN-01 maintains a process-local registry `Map<String, ChannelAdapter>` keyed by `adapter_impl`. The instance was already `init()`'d at boot time (or at last config-reload) with `config_jsonb` and `secretResolver`. If `adapter_impl` is not registered → delivery `TERMINALLY_FAILED` with `failure_reason = ADAPTER_NOT_REGISTERED` (per R-ICN-01-D-17).
- 4. Render the message.** Pull the `staff_notification_template` row for (`operator_code`, `template_code`, `channel`), substitute `template_variables` into the body (Mustache style). Missing required variables → emit `StaffNotificationTemplateRenderWarning`, render as «MISSING:varname» (per R-ICN-01-D-13).
- 5. Read the per-user channel identity.** Query `staff_notification_user_channel_identity` for (`recipient_user_id`, `channel`):

```

SELECT identity_jsonb
FROM staff_notification_user_channel_identity
WHERE user_id = $1 AND channel = $2;

```

The result (or null if no row) is wrapped into a `RecipientInfo` along with the `user_id` and a lazy fallback `UserProfile` supplier that, when invoked, calls `FOUNDATION_AUTH GET /api/users/{user_id}`.

- 6. Call adapter.dispatch(deliveryRow, renderedMessage, recipientInfo).** The adapter reads `recipientInfo.identityJsonb`. If it has what it needs, it dispatches. If `identityJsonb` is null AND the adapter supports fallback, it invokes `recipientInfo.fallbackProfileLookup.get()` to fetch the user's Keycloak profile, derives the channel identifier from there, and proceeds (optionally backfilling the identity row for next time). If the adapter cannot derive an identifier → returns `ChannelDispatchResult(success=false, failureReason=IDENTITY_UNRESOLVABLE)`.

7. **Update the delivery row.** Adapter result → `staff_notification_delivery.status` (DISPATCHED / FAILED), `provider_message_id`, `provider_response`, `attempts++`, `last_attempt_at`. On FAILED, schedule retry per `staff_notification_channel_config.retry_backoff_seconds` (per R-ICN-01-D-7).
8. **Emit Kafka event.** `StaffNotificationDelivered` on success, `StaffNotificationDeliveryFailed` on `TERMINALLY_FAILED`.

This 8-step path runs once per delivery row (one notification with 4 recipients × 3 channels = 12 invocations of this path, executed in parallel by the worker pool). Each step is idempotent — a worker crash mid-path resumes safely on the next worker pickup because the delivery row's status remains PENDING until step 7 commits.

7.1 EMAIL — v1.0 adapter implementations

Two ship in v1.0:

`smtp-classic` — Jakarta Mail to a configured SMTP relay. Used by WIK / WUG / WTZ.

`config_jsonb` shape:

```
{
  "host":          "smtp-relay-internal.wik.sophix.local",
  "port":          587,
  "starttls":      true,
  "from":          "sophix-noreply@wik.sophix.local",
  "auth_vault_ref": "secret/icn/wik/smtp",
  "bounce_poll_mailbox": "bounce-monitor@wik.sophix.local",
  "bounce_poll_vault_ref": "secret/icn/wik/smtp-bounce-imap"
}
```

Bounce-back handling: SMTP relay forwards bounces to the configured bounce mailbox; a bounce-poller worker reads it (IMAP) every 5 minutes and marks corresponding `staff_notification_delivery` rows as `TERMINALLY_FAILED` with `failure_reason = BOUNCE`.

`microsoft-graph-mail` — Microsoft Graph API `sendMail` endpoint. Used by YASSN (Office 365 tenant).

`config_jsonb` shape:

```
{
  "tenant_id":      "yassn-tenant-uuid",
  "app_id":         "yassn-graph-app-id",
  "auth_vault_ref": "secret/icn/yassn/graph",
  "from":          "sophix-noreply@yassn.sophix.local"
}
```

OAuth client-credentials flow with the app secret resolved from Vault on init; token refresh handled by the adapter (Graph tokens are 60-min). Bounce signal via Graph `getMessage` callbacks isn't ideal; adapter polls a delivery-report mailbox similar to SMTP for v1.0.

Future: `aws-ses`, `mailgun-api`, `postmark-api`, `sendgrid-api` — drop-in additions when an operator wants them.

Recipient identity resolution (both adapter implementations): consume `identity_jsonb.address` from `staff_notification_user_channel_identity` if present. If null, invoke `recipient.fallbackProfileLookup()` → `FOUNDATION_AUTH GET /api/users/{userId}` → read email attribute from Keycloak profile → use that. On success, MAY backfill an identity row with `source='auto-resolved-from-auth'`. If both paths yield no email → return `IDENTITY_UNRESOLVABLE`.

7.2 SLACK — v1.0 adapter implementations

`slack-bot-api` — Slack Web API `chat.postMessage`. Used by WIK / WUG.

`config_jsonb` shape:

```
{
  "workspace_id":      "T7K1S0PH1XKE",
  "bot_token_vault_ref": "secret/icn/wik/slack/bot-token",
  "base_url":          "https://slack.com/api",
  "interactivity_callback": "https://api.wik.sophix.com/api/staff-notifications/slack-interactivity"
}
```

Recipient identity resolution: consume `identity_jsonb.slack_user_id` from `staff_notification_user_channel_identity` if present. If null, invoke `recipient.fallbackProfileLookup()` → derive email from Keycloak → call `Slack.users.lookupByEmail` against the workspace → on success, MAY backfill the identity row with `source='auto-resolved-from-auth'`. If both paths yield no Slack ID → delivery `TERMINALLY_FAILS` with `failure_reason = IDENTITY_UNRESOLVABLE`.

Card format: Slack block kit with action button "Open in BO UI" pointing at the deeplink. Button click flows through Slack's interactivity callback URL → the adapter's interactivity endpoint → ICN-01 ACK endpoint.

Future: `slack-connect-api` for cross-workspace / shared-channel scenarios.

7.3 MSTEAMS — v1.0 adapter implementations

`msteams-incoming-webhook` — Microsoft Teams Incoming Webhook. Used by YASSN.

`config_jsonb` shape:

```
{
  "channel_webhooks": {
    "yassn-kyc-reviews": "https://outlook.office.com/webhook/aaa.../IncomingWebhook/.../...",
    "yassn-finance-approvals": "https://outlook.office.com/webhook/bbb.../IncomingWebhook/.../...",
    "yassn-noc": "https://outlook.office.com/webhook/ccc.../IncomingWebhook/.../..."
  },
  "vault_ref": "secret/icn/yassn/msteams-webhooks",
  "candidate_group_to_channel": {
    "yassn-l1-kyc": "yassn-kyc-reviews",
    "yassn-kyc-leaders": "yassn-kyc-reviews",
    "yassn-finance-approvers": "yassn-finance-approvals",
    "yassn-noc-team": "yassn-noc"
  }
}
```

Limitation: Incoming Webhooks are channel-scoped, not user-scoped — a posted message goes to a Teams channel, not to a specific person's DM. The adapter uses `candidate_group_to_channel` to pick the right Teams channel based on the notification's candidate group. All members of that Teams channel see the message. This means per-user channel preferences (`staff_notification_user_pref`) for MSTEAMS don't gate delivery in this adapter — they only gate whether MSTEAMS is in the user's effective channel list. Once selected, all candidate-group members see the same Teams post.

Future: `msteams-graph-channel` — uses Microsoft Graph to post to specific channels (more flexible) or to user DMs (`chat.create` + `chat.sendMessage`) for per-user notification. Migration path is the same as any binding swap.

7.4 IN_APP_PUSH — v1.0 adapter implementation

`inapp-websocket-fanout` — the BO UI maintains a WebSocket connection to ICN-01's push endpoint after login.

`config_jsonb` shape:

```
{
  "ws_endpoint": "wss://push.wik.sophix.com/staff",
  "polling_fallback_endpoint": "https://push.wik.sophix.com/staff/inbox",
  "session_jwt_audience": "icn-01",
  "session_ttl_seconds": 3600,
  "buffer_unack_hours": 72
}
```

ICN-01 fans out the notification to all active sessions for the recipient. The adapter uses `user_id` directly to look up active sessions in its in-memory session registry — `identity_jsonb` is not consulted for routing. If a `staff_notification_user_channel_identity` row exists for (`user_id`, `IN_APP_PUSH`), it's an informational marker that this user is set up to receive in-app pushes; if absent, dispatch still proceeds.

If no active session, the delivery row stays PENDING (subject to retry policy in R-ICN-01-D-7); on session establishment, the BO UI subscribes "give me my unACKed notifications for the last N hours" via the polling fallback endpoint and catches up. After `ack_window_hours` of no session, the row goes SUPPRESSED — not TERMINALLY_FAILED, because the user wasn't reachable on this channel, but it's not the channel's fault.

WebSocket auth: JWT in the connection upgrade handshake; verified against `FOUNDATION_AUTH` on connect.

Future: `inapp-server-sent-events` (alternative transport for environments where WebSocket is hard); `inapp-firebase-cloud-messaging` (if a native Wananchi staff mobile app exists).

7.5 Indirection consequences

The adapter binding pattern means:

- **Operator choice is decoupled from code.** YASSN choosing Microsoft Graph for email instead of SMTP doesn't require a code branch — it just inserts a different binding row.
- **Provider migration is data-only.** Swapping SMTP for Graph mid-flight is an UPDATE on the binding row plus possibly a parallel-run period (two bindings briefly coexisting, one enabled=false). Zero domain-config change.

- **Future channels land cleanly.** Adding WhatsApp Business: write a new adapter implementing ChannelAdapter, add channel value WHATSAPP_BUSINESS to the templates + bindings, insert config. No schema migration on the core ICN-01 tables.
- **Multi-operator deployments share code.** WIK and WUG both use smtp-classic for EMAIL — same Java class, different config_jsonb rows.
- **Testing in production is safe.** New adapter implementations can be deployed and inserted as enabled=false bindings, allowing dry-runs or canary tests without affecting live dispatch.

8. Stub retirement

ICN-01 retires the `staff_notification_stub` table that lives in FUL-02 FRAMEWORK §6.3 and is referenced by WO module FRAMEWORK §9.1.

8.1 Mapping `staff_notification_stub` → ICN-01

The stub's 8-column schema maps to the full module:

Stub column	Maps to	Notes
<code>notification_id</code>	<code>staff_notification.notification_id</code>	Same key, same ULID format. Direct migration.
<code>order_id</code>	<code>staff_notification.source_business_key</code>	The stub assumed all notifications were order-related; ICN-01 generalises to any business key. For migrated rows, <code>source_module='FUL-02'</code> .
<code>task_id</code>	<code>staff_notification.source_task_id</code>	Direct migration.
<code>candidate_group</code>	<code>staff_notification.candidate_group</code>	Direct migration.
<code>payload_jsonb</code>	<code>staff_notification.template_variables</code> + the calling-module rebuilds the <code>templateCode</code> reference	The stub's <code>payload_jsonb</code> was the full HTTP-connector payload. Migration synthesises a <code>template_code</code> based on the order's last user-task context (best-effort for historical rows).
<code>dispatch_method</code>	retired	The stub only had <code>INTERNAL_SMTP</code> . ICN-01 expands to 4 channels; migrated stub rows get one <code>staff_notification_delivery</code> row with <code>channel='EMAIL'</code> .
<code>dispatch_status</code>	<code>staff_notification_delivery.status</code>	<code>DISPATCHED</code> → <code>DISPATCHED</code> ; <code>FAILED</code> → <code>TERMINALLY_FAILED</code> . No ACK history in stub; migrated rows lose ACK information.
<code>created_at</code>	<code>staff_notification.created_at</code>	Direct migration.

8.2 Migration steps

1. **Pre-deployment:** ICN-01 schema deployed alongside `staff_notification_stub`. Both tables coexist during transition.
2. **Backfill:** One-time job iterates `staff_notification_stub` rows. For each:
 - Insert `staff_notification` row with `source_module='FUL-02'` (or `'WO-01'` per source-table inspection), `template_code='stub-migrated-historical'` (a generic template), `template_variables={"originalPayload": <payload_jsonb>}`.
 - Insert one `staff_notification_delivery` row per stub row with `channel='EMAIL'`, status mapped per above.
 - Set `staff_notification.status='EXPIRED'` for all historical rows (we don't re-dispatch).
3. **Caller cutover:** FUL-02's `NotificationHandlerResource` (§3 of that DD) and WO module's `NotificationHandlerResource` (FRAMEWORK §6) switch their `INSERT INTO staff_notification_stub` calls to `POST /api/staff-notifications`. The HTTP connector contract from Camunda is unchanged — only the handler's internal implementation changes.
4. **Verification:** Run for 2 weeks with both systems updated dual-write style; reconcile daily by comparing `staff_notification_stub` row counts to `staff_notification` row counts created via the new path; investigate any deltas.
5. **Removal:** Drop `staff_notification_stub` table. Remove FUL-02 FRAMEWORK §6.3 and WO module FRAMEWORK §9.1 schema documentation; replace with "retired — see DD_ICN-01" reference.

8.3 Caller-side changes

FUL-02 FRAMEWORK §3 step 4 currently reads:

4. v1.0 dispatch: write a row to `staff_notification_stub` with `dispatch_method = INTERNAL_SMTP`; send minimal email via internal SMTP wrapper.

Post-ICN-01:

4. Dispatch: call `POST /api/staff-notifications` on ICN-01 with `templateCode` corresponding to the user-task type (e.g., `kyc-l1-approval-needed`), passing `templateVariables` derived from the user-task's business context. ICN-01 returns 202 with `notification_id`; the handler stores it on the order's audit trail for traceability.

WO module FRAMEWORK §9.1 is updated equivalently.

A minor caller-side change: each calling module now picks a `template_code` per user-task type. FUL-02 step satellites each declare their template code in their `templateCodes` config (e.g., STEP-KYC declares `kyc-l1-approval-needed` and `kyc-final-approval-needed`). This is a small one-time refactor of the existing satellites — no schema impact.

9. Workshop sources

There is no dedicated workshop deep-dive for Internal Communications. The functional area is implicit in many other workshops:

- **Customer Management workshop** (deep-dive 01, 2026-04-27 / 2026-04-28) — described two-level KYC approval (L1 supervisor → Team Leader), implying staff notification fan-out to each level.
- **Work Order workshop** (implicit in deep-dive 07) — contractor dispatcher notifications, NOC notifications, tech-claim-pending alerts.
- **Dunning workshop** (implicit in deep-dive 04) — L3 escalation alerts to internal reviewers, OutageCredit batch completion alerts to Finance.
- **Refund / adjustment workflows** — Finance approval requirements imply approval-needed staff notifications.

V3 design intent for ICN-01: **provide a single, multi-channel internal notification fabric** so that staff can pick the channel they actually monitor, with fall-back across channels if one fails. This is a V3 capability not present in the BroadHub-era TZ codebase, which (per memory rule #17, awaiting cartography confirmation) appears to use email-only via internal SMTP — the same pattern `staff_notification_stub` reproduces today.

Out of v1.0 scope:

- **WhatsApp Business for staff** — flagged for UG/TZ workshops; UG specifically may want this. Catalog and adapter extension is straightforward (new channel kind + new adapter); not in v1.0.
- **SMS to on-call staff** — feasible via DD_NOT-01's customer SMS gateway shared as infra, but the cross-module reuse is out of v1.0 scope.
- **Voice call escalation** (PagerDuty-style telephony for critical incidents) — out of scope.
- **Native mobile push** (iOS/Android push to a staff mobile app) — there's no Wananchi staff mobile app today; if one is built, push integration is a future enhancement.

10. Cross-DD coordination

10.1 Module owner

ICN-01 is owned by the Foundations team (Group I) alongside ILM-CFG-01, PLM catalogs, RLM-CFG-01, SIP-01. It's a peer-of dependency for all workflow modules.

10.2 Upstream callers

Caller	Endpoint called	When
FUL-02 NotificationHandlerResource	POST /api/staff-notifications	Every Camunda user task in FUL-02 (KYC, payment review, install confirmation, etc.)

Caller	Endpoint called	When
WO module NotificationHandlerResource	POST /api/staff-notifications	Contractor dispatcher assign-needed, tech-claim-pending, finalize-confirmation, NOC verification
BIL-04 Dunning Engine	POST /api/staff-notifications	L3 escalation alerts to internal reviewers (the workflow direct-calls — not via Camunda user task; BIL-04 is rule-driven)
BIL-05 Wallet refund workflow	POST /api/staff-notifications	Refund approval needed (Finance candidate group)
Future: NMS-MONITORING	POST /api/staff-notifications	Critical outage alerts to NOC; provisioning failure alerts
Future: ASR-* (Ticketing)	POST /api/staff-notifications	Ticket-assigned alerts, SLA-breach-imminent alerts

10.3 Downstream consumers (Kafka)

Module	Event consumed	What they do
Ops dashboards / Grafana	All <code>sophix.icn.*</code> topics	Display dispatch + ACK metrics, channel-success rates
CVM analytics	StaffNotificationCreated, StaffNotificationAcknowledged	Staff-productivity analytics (mean time to ACK by role/team)
PagerDuty / on-call	StaffNotificationFailedToReachAnyone	Page on-call ops
Source-module dev teams	StaffNotificationTemplateRenderWarning	Detect their own template-variable bugs
Future BO UI alerting	StaffNotificationAcknowledged	Real-time inbox refresh in the UI (alternative to polling)

10.4 Sibling modules (peer dependencies)

Module	Direction	Contract
FOUNDATION_AUTH	ICN-01 ← AUTH	GET /api/groups/{group}/members?operatorCode=<> to resolve candidate group; GET /api/users/{userId} to fetch user contact details. JWT verification on all incoming requests.
DD_NOT-01 (Customer Notifications)	strictly disjoint	DD_NOT-01 handles customer notifications; ICN-01 handles staff notifications. No shared code path. Separate Kafka topic namespaces (<code>sophix.icn.*</code> vs <code>sophix.not.*</code>). Separate database schemas. A future shared "notification gateway" layer is conceivable but explicitly out of v1.0 scope to preserve the staff/customer separation.
FOUNDATION_KAFKA	ICN-01 → Kafka	Topics created under <code>sophix.icn.*</code> namespace. Standard partition strategy (by operatorCode).
Vault	ICN-01 ← Vault	Secret references stored inside <code>staff_notification_adapter_binding.config.jsonb</code> (e.g., <code>"auth_vault_ref": "secret/icn/wik/smtp"</code>). Adapters resolve them at adapter init via the <code>SecretResolver</code> interface. Reload on rotation (adapter receives a re-init signal when a binding row is updated).

10.5 Pre-existing stubs retired

Stub	Owner module	Status
<code>staff_notification_stub</code>	FUL-02 FRAMEWORK \$6.3	Retired by this DD (\$8)
<code>staff_notification_stub</code> (same name, mirrored)	WO module FRAMEWORK \$9.1	Retired by this DD (\$8) — both modules cut over to ICN-01 simultaneously

10.6 Dependencies (build order)

1. **FOUNDATION_AUTH** must be operational with group-membership API + user-detail API.
2. **FOUNDATION_KAFKA** must be operational with the `sophix.icn.*` topic prefix authorised.
3. **Vault** must be operational with channel-adapter secret paths populated.

4. **Per-operator channel provider infrastructure** must be provisioned per the operator's chosen adapter implementations: e.g., WIK provisions an internal SMTP relay + Slack bot + BO UI WebSocket endpoint; YASSN provisions a Microsoft Graph app registration + MS Teams Incoming Webhooks. Choice of provider is captured in `staff_notification_adapter_binding` rows.
5. **Adapter binding rows** must be seeded for each (`operator_code`, `channel`) the operator enables. Sample seed scripts ship per operator (`bindings-seed-wik.sql`, `bindings-seed-yassn.sql`, etc.). Vault secret paths must be pre-populated before bindings are activated.
6. **Template seed** must be loaded for the operator before any workflow that calls ICN-01 starts using it. v1.0 deployment includes a `templates-seed.sql` script loading the ~30 KE WIK templates.

11. Runtime walkthrough — narrative

One scenario: Jane Mwangi's KYC L1 approval, from the moment FUL-02's Camunda workflow parks at the user task, through ICN-01 dispatch fan-out, to the supervisor's ACK.

11.1 Workflow parks at user task

Day 0, 17:30 — FUL-02 STEP-KYC Camunda BPMN reaches the `await-l1-kyc-approval` user task. Camunda enters the task; the BPMN's HTTP connector fires:

```
POST https://api.wik.sophix.com/api/notify/kyc-approval-needed HTTP/1.1
X-Camunda-Connector-Secret: <shared-secret>
Content-Type: application/json

{
  "taskId":      "cam_task_l1_01HZ_JANE",
  "taskName":    "await-l1-kyc-approval",
  "businessKey": "ord_01HX_JANE_TRIPLE_PLAY",
  "orderId":     "ord_01HX_JANE_TRIPLE_PLAY",
  "customerId":  "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "operatorCode": "WIK",
  "candidateGroup": "kenya-l1-kyc",
  "channels":     ["email", "slack", "in_app_push"],
  "template":     "kyc-l1-approval-needed",
  "urgency":      "medium",
  "franchiseCode": "KE-NRB-FRANCHISE-04"
}
```

FUL-02's `NotificationHandlerResource` receives this, validates the shared secret, then builds the ICN-01 dispatch request and calls:

11.2 Dispatch enters ICN-01

```
POST /api/staff-notifications HTTP/1.1
Authorization: Bearer <ful-02-service-account-jwt>
Idempotency-Key: ful-02-kyc-l1-cam_task_l1_01HZ_JANE
Content-Type: application/json

{
  "operatorCode":      "WIK",
  "sourceModule":       "FUL-02",
  "sourceTaskId":       "cam_task_l1_01HZ_JANE",
  "sourceBusinessKey":  "ord_01HX_JANE_TRIPLE_PLAY",
  "candidateGroup":     "kenya-l1-kyc",
  "templateCode":       "kyc-l1-approval-needed",
  "templateVariables": {
    "customerName":     "Jane Mwangi",
    "orderId":           "ord_01HX_JANE_TRIPLE_PLAY",
    "franchiseCode":     "KE-NRB-FRANCHISE-04",
    "deepLinkUrl":       "https://bo.wik.sophix.com/tasks/cam_task_l1_01HZ_JANE"
  },
  "urgency":            "medium",
  "ackWindowHours":     24
}

→ 202 Accepted
{
  "notificationId":      "notif_01HZ_KYC_L1_JANE",
  "expectedRecipients":  4,
  "createdAt":           "2026-02-16T17:30:00Z",
  "expiresAt":           "2026-02-17T17:30:00Z"
}
```

FUL-02 records `notif_01HZ_KYC_L1_JANE` on the order's audit trail and returns 200 to the Camunda connector.

11.3 ICN-01 resolves recipients

ICN-01's request handler synchronously:

1. Validates the JWT (FUL-02 service account is recognised; operator scope matches).
2. Validates the template exists: (`WIK`, `kyc-l1-approval-needed`) resolved to 3 channel variants (`EMAIL`, `SLACK`, `IN_APP_PUSH`).
3. Calls `FOUNDATION_AUTH: GET /api/groups/kenya-l1-kyc/members?operatorCode=WIK` → returns 4 supervisor user IDs.
4. Inserts `staff_notification` row: `notif_01HZ_KYC_L1_JANE`, `status=PROCESSING`, `expected_recipients=4`, `expires_at=2026-02-17T17:30:00Z`.
5. For each of the 4 supervisors:
 - Reads `staff_notification_user_pref` if exists; else uses operator default channel config.
 - Computes effective channel list per `R-ICN-01-D-4` (priority order × enabled × not-suppressed × quiet-hours-honoured).
 - For each channel in the list: inserts a `staff_notification_delivery` row with `status=PENDING`.
6. Returns 202 to FUL-02.

By the end of step 6: 4 supervisors × ~3 channels = ~12 delivery rows inserted (some channels may be SUPPRESSED if a supervisor opted out — those rows skip insertion or insert as SUPPRESSED).

11.4 Async dispatch

A background worker pool picks up each PENDING delivery row and runs the 8-step dispatch resolution from §7.0. Briefly, for each row:

1. Reads the `staff_notification_adapter_binding` for (`operator_code`, `channel`) — for `WIK`, this resolves to: `EMAIL` → `smtp-classic`, `SLACK` → `slack-bot-api`, `IN_APP_PUSH` → `inapp-websocket-fanout`.
2. Looks up the registered `ChannelAdapter` instance (already init'd at boot from the binding's `config_jsonb`).
3. Reads `staff_notification_user_channel_identity` for the recipient × channel — for supervisor `franchise_KE-NRB-04`'s `EMAIL` delivery, this returns `{"address": "sup_NRB04@wik.wananchi.com"}`; for `SLACK`, `{"slack_user_id": "U04K1S0PH1XKE"}`; for `IN_APP_PUSH`, the empty `{}` row (adapter uses `user_id` directly anyway).
4. Invokes `adapter.dispatch(deliveryRow, renderedMessage, recipientInfo)` with `recipientInfo` containing the `identity_jsonb` + a lazy fallback profile lookup.
5. Records the result on the delivery row.

For Jane's notification, this fans out to:

- 4 × `IN_APP_PUSH` via `inapp-websocket-fanout`: `WebSocket` fanout to each supervisor's session. For 3 supervisors logged into BO UI at this moment, message arrives instantly; their tray badge increments. For 1 supervisor not logged in, delivery stays PENDING (no retry on the `WebSocket` side; waits for next BO UI session per §7.4).
- 4 × `SLACK` via `slack-bot-api`: posts the formatted block-kit card to each supervisor's Slack DM. All 4 succeed; delivery rows → `DISPATCHED` with `provider_message_id` = Slack ts.
- 4 × `EMAIL` via `smtp-classic`: SMTP relay accepts all 4; delivery rows → `DISPATCHED` with `Message-ID`.

If YASSN had received the same notification, `EMAIL` would have routed through `microsoft-graph-mail` instead, calling Graph's `sendMail` endpoint — same delivery row shape, different adapter implementation. The choice was made at the binding layer, transparent to the workflow + transparent to the rest of ICN-01.

ICN-01 emits `StaffNotificationCreated` once and `StaffNotificationDelivered` per successful delivery (10 deliveries × 1 event each = 10 events on `sophix.icn.notification-delivered`).

11.5 Supervisor acknowledges

Day 1, 08:15 — supervisor `franchise_KE-NRB-04` logs into BO UI in the morning. Their tray shows 3 pending notifications including Jane's KYC. They click the notification, which deep-links to the user task page. The BO UI then calls:

```
POST /api/staff-notifications/notif_01HZ_KYC_L1_JANE/ack HTTP/1.1
```

```

Authorization: Bearer <supervisor-jwt>

{
  "recipientUserId": "supervisor_franchise_KE-NRB-04",
  "ackChannel":      "IN_APP_PUSH",
  "ackContext":      { "userAgent": "B0-UI/2.4.1", "viewSource": "deeplink" }
}

→ 200 OK
{
  "notificationId": "notif_01HZ_KYC_L1_JANE",
  "status":         "ACKNOWLEDGED",
  "acknowledgedBy": "supervisor_franchise_KE-NRB-04",
  "acknowledgedAt": "2026-02-17T08:15:00Z"
}

```

ICN-01 atomically:

1. Updates the supervisor's IN_APP_PUSH delivery row to ACKNOWLEDGED.
2. Updates the parent staff_notification to ACKNOWLEDGED + acknowledged_by + acknowledged_at.
3. Marks the other supervisors' still-PENDING (i.e., not-yet-dispatched) deliveries as SUPPRESSED.
4. Emits StaffNotificationAcknowledged on sophix.icn.notification-acknowledged (the event payload from §6.2).

The supervisor's BO UI is now on the KYC L1 review screen for Jane's order. They review the documents, click "Approve". The BO UI calls FUL-02's own POST /api/orders/{orderId}/user-tasks/{taskId}/complete endpoint with the approval decision. FUL-02 forwards to Camunda's POST /engine-rest/task/{id}/complete, the workflow advances. **ICN-01 is not involved in the completion path** — it's a transport layer only; the workflow's actual state machine is owned by Camunda + FUL-02.

11.6 Lifecycle summary

Within ICN-01, this notification leaves the following audit trail:

- 1 staff_notification row, status=ACKNOWLEDGED, time-to-ACK = ~15 hours (overnight wait is normal).
- ~10 staff_notification_delivery rows across 4 supervisors × multiple channels. 1 ACKNOWLEDGED (the IN_APP_PUSH for supervisor_franchise_KE-NRB-04). Several DISPATCHED (Slack + email to all 4). Some SUPPRESSED (the rest, after ACK).
- 1 StaffNotificationCreated event.
- ~10 StaffNotificationDelivered events.
- 1 StaffNotificationAcknowledged event.

Total: one staff member needed action, three channels reached them, fan-out to peer supervisors was suppressed once one acted. Net effect: the team isn't drowned in duplicate notifications, but redundancy was present in case the first supervisor was unavailable.

End of DD_ICN-01-Internal_Communications-v1.0.